

CODE SPORT



In this month's column, we will first briefly discuss the common patterns in coding interview questions, and then feature a few of these questions.

There are typically a number of patterns in coding interview questions. Some of these include dynamic programming, sliding window, monotonic queue/stack, divide and conquer patterns, etc. Problems that appear difficult at first, turn out to be quite simple once the underlying pattern can be identified. Let us talk about a few examples of each of these patterns.

Dynamic programming (DP) continues to be quite popular in coding interviews. Dynamic programming questions are difficult unless you have practised earlier on similar types of questions. While the solution once framed into a DP equation looks quite simple, there is quite a leap from the problem statement to framing the dynamic programming solution equation. One way to navigate the gap, typically, is by thinking about the problem first in terms of a top-down recursive solution. Once we can find the recursive equation, extend the solution by applying memorisation so that you don't have to recompute the same sub-problems over and over again. Once we have the memorised solution, try and see if you can frame it as a DP equation by looking at the sub-problems. Let us look at a sample question.

There is a frog that needs to cross a river. There are stones on the crossing path, which are divided into units of one, on which the frog can jump and cross the river. The

position of these stones is given as integers in a sorted array. The frog is sitting on the first stone, which is at position 0, and the first jump must be of one unit. At any point during the journey, when the frog jumps, it needs to only land on a stone, else it can't cross the river. At any stone, it can only jump $(k-1)$, k or $(k+1)$ units from the current stone, where k is the size of the previous jump. The frog needs to land on the last stone for it to cross the river. You need to write code to determine whether the frog can actually cross the river, given an array representing the position of the stones on the path.

The brute force solution would be to frame this problem as a recursive solution. Given the position of the current stone, and the size of the last jump, we can recursively keep checking whether we can reach a stone from the current position, and from then onwards to the next stone, and so on, until we end up at the last stone or we run out of stones. Given the size of the last jump, k , the possible sizes of the new jump are $(k-1)$, k and $(k+1)$. We need to check if there exists a stone at the current position + the new jump size. If so, we can then again recursively do the check from that stone (with that stone as the starting point and the jump size with which that stone was reached). If we can reach the last stone through any of the paths, the crossing was successful. If there is no successful path to the last stone, the frog fails to cross.

